Project 2 — Authoring Kernels

CS 6959 · Ray Tracing Hardware
TRaX color gradient and Mandelbrot escape-time rendering

A. Setup and required questions

The simulator was freshly cloned from Utah-Graphics-Lab/arches at commit `97bebdc` and built in Release mode. The supplied RISC-V toolchain was installed under `/opt/riscv` (GCC 15.2.0). Kernels target `rv64imaf`, ABI `lp64f`, with `-nostartfiles -emain -mno-relax -O3 -ffp-contract=off`. The last flag keeps operation boundaries consistent with the native CPU validation; no hardware or simulator source was modified.

1. How long did the color-gradient simulation take?

For 256 × 256 pixels, the simulator printed **Simulation time: 1 s** (its simulation-loop timer is rounded to whole seconds). A separate Python `time.perf_counter()` measurement around the simulator subprocess gave **0.865166 s** end-to-end, including scene initialization and PNG output, but excluding compilation. These are different timing scopes, and this is one measured run rather than an average. The modeled hardware executed **8,874 cycles at 1,515 MHz**, corresponding to **0.00586 ms** of simulated frame time; this is not the host runtime.

2. What color format is written to the framebuffer?

Each pixel is a 32-bit integer in `0xAABBGGRR` format: bits 0–7 are red, 8–15 green, 16–23 blue, and 24–31 alpha. In little-endian memory the byte order is RGBA. The gradient uses alpha 255, blue 0, and position-dependent red and green. Arches vertically flips the framebuffer when writing the PNG.

3. What is an execution model, and what constitutes TRaX's model?

An execution model defines the software-visible rules for how work executes, how threads are identified and grouped, and how they interact. TRaX's model consists of **threads and tiles**. A logical thread is identified by a work ID; a tile groups 32 consecutive IDs in the default configuration. A thread stays on its TP, and a tile's threads execute within one TM. TPs and TMs are the microarchitecture implementing these rules, not the software grouping itself. TRaX uses SPMD with independent program counters, rather than requiring all threads in a tile to execute in lockstep.

4. What purpose does `fchthrd()` serve?

`fchthrd()` atomically obtains a unique next work ID while the scheduler allocates tiles to TMs, letting available execution contexts claim more work. Calling it again does not reset local program state, so per-pixel state must be explicitly reinitialized for every new ID.

Figure 1: Gradient: all 65,536 pixels written.

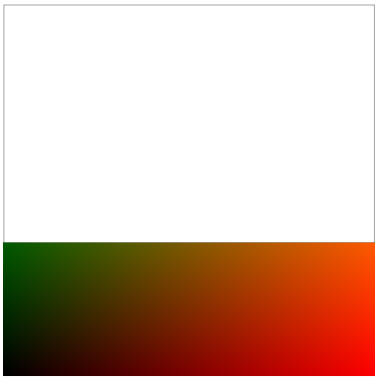


Figure 2: Without the loop: 23,552 pixels written.

The single-pass control retains a bounds check but removes repeated work fetching. Exactly $46 \times 64 \times 8 = 23,552$ pixels have alpha 255, matching the launch-time hardware context count. The remaining 41,984 pixels are transparent. The loop is needed to process all 65,536 pixels.

B. Custom kernel: Mandelbrot

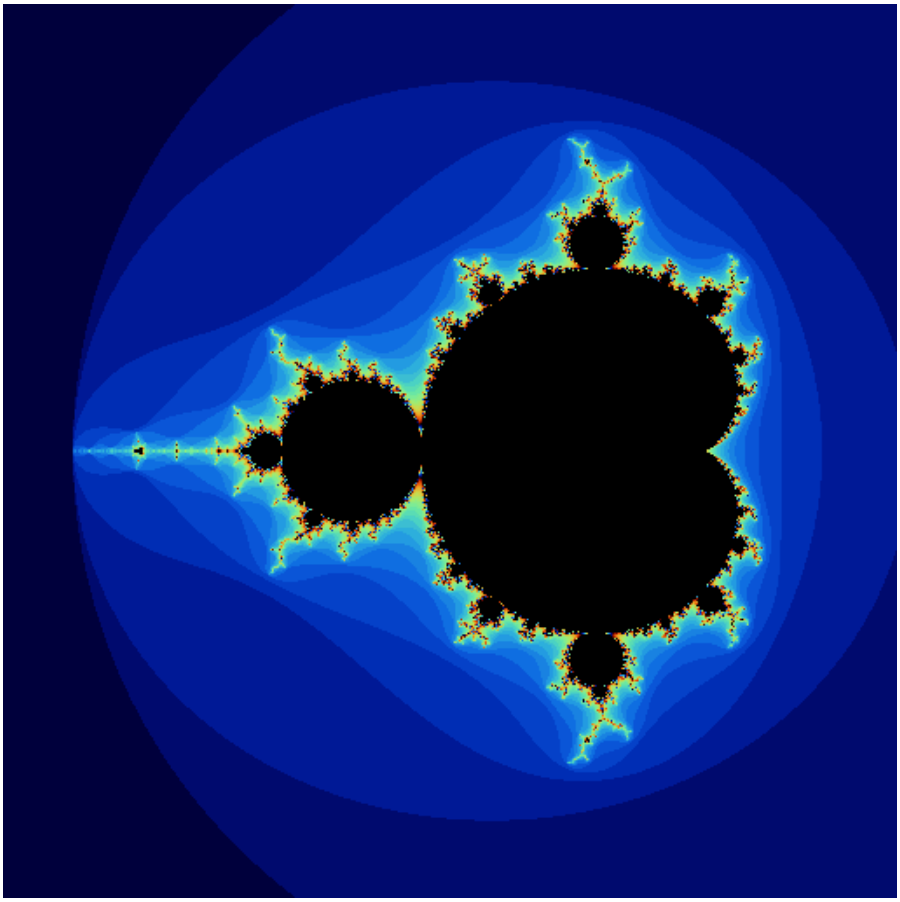


Figure 3: Final submitted image: 512 × 512 pixels, 96 iterations maximum.

Algorithm. Each work ID maps to a unique row-major pixel with `x = tid % width` and `y = tid / width`. Pixel centers map into the complex plane with scale `3.2f / width`, centered at real coordinate −0.65. For the square image, this covers approximately $[-2.25, 0.95] \times [-1.6, 1.6]$. Using the same scale on both axes also preserves aspect ratio for rectangular images.

Starting with $z_0 = 0$, the kernel repeatedly evaluates $z_{n+1} = z_n^2 + c$ until $|z|^2 > 4$ or 96 iterations are reached. Points that reach the cap are black; this is a finite-iteration approximation, not a proof of membership in the Mandelbrot set. Escaping points receive a repeating polynomial RGB palette based on the iteration count modulo 32. The visible bands are intentional; no logarithmic smoothing is used.

Parallel correctness. Every ID writes only its own framebuffer element. The complex state and iteration counter are local to each pixel calculation and reset on every loop iteration. There are no dynamic allocations, mutable globals, static variables, exceptions, double-precision calculations, or scene-data accesses in the kernel. Different pixels need different iteration counts; contexts that finish one pixel can request another ID. No performance advantage over another scheduling policy is claimed without a comparison.

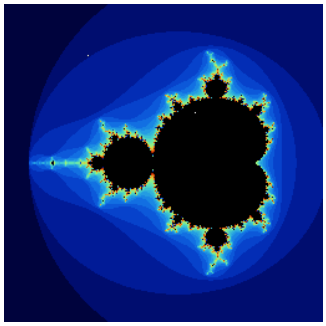
Measured final run. The 512 × 512 simulation took **2.757983 s** end-to-end; the simulator printed **2 s** for its rounded simulation-loop timer. It executed **33,755 cycles**, or **0.0223 ms** of modeled frame time. All 262,144 pixels match the native C++ reference byte-for-byte. Two further runs produced the same PNG and cycle count, with zero pixel mismatches.

C. Validation, reproducibility, and limitations

Run	Cycles	Host wall time	Pixel check
Gradient, 256^2	8,874	0.865166 s	65,536 / 65,536
Single pass, 256^2	5,089	0.594484 s	23,552 written
Mandelbrot, 512^2	33,755	2.757983 s	262,144 / 262,144

The gradient is checked against its analytic integer-channel values, accounting for the vertical PNG flip. The single-pass image is checked for exactly the first 23,552 row-major pixels and transparent zeros everywhere else. The Mandelbrot reference compiles the exact per-pixel C++ function natively with matching optimization and floating-point contraction settings and compares all four channels of every pixel. This tests simulator execution and write coverage; it is not an independently implemented mathematical oracle. A native AddressSanitizer/UndefinedBehaviorSanitizer run at 257 × 223 also completed without diagnostics. ELF inspection confirmed RISC-V ELF64, little-endian, and the single-float ABI.

Exploratory failure retained, not retouched



The initial 256 × 256 Mandelbrot trial had **two transparent pixels** instead of the CPU reference colors. L1 recorded 65,536 uncached requests, but DRAM recorded only 65,534 stores. This is consistent with a suspected simulator write-drain/termination issue, not established here as a proven root cause. Its original image, log, binary, and failed validation are retained under `exploratory/`; it is not counted as a passing result.

No waiting loops, image postprocessing, or simulator changes were added to hide this anomaly. The final 512 × 512 configuration was separately validated in three runs, all with 262,144 DRAM stores and exact CPU agreement. This does not prove correctness at every other resolution.

Reproducing the submitted run

Build the simulator in Release mode, put `/opt/riscv/bin` on `PATH`, then run:

```
cd /root/arches/src/trax-kernel
make
mkdir -p /root/project2/reproduced
cd /root/project2/reproduced
/root/arches/build/src/arches-v2/arches-v2 \
  --arch-name=TRaX --dataset-dir=/root/datasets \
  --scene-name=teapot --framebuffer-width=512 \
  --framebuffer-height=512 --pregen-rays=0 \
  --logging-interval=100000 > trax_log.txt 2>&1
```

The host still initializes a scene, so the small teapot dataset is provided even though the kernel never reads it. The upstream material loader reports an unsupported material line; this does not affect this data-independent kernel. Zero-ray statistics include undefined ratios (`nan`); ray throughput and power fields are not used as meaningful metrics for this project.

Submission files: `main.cpp`, the corresponding `trax_log.txt`, `out.png`, and this `report.pdf`. The repository additionally contains `run.py`, `validate.py`, the native reference harness, source/binary SHA-256 metadata, and original logs and images. The supplied toolchain and course handout are not redistributed in the source repository.